PROG7312 — Programming 3B

Final Portfolio of Evidence — Implementation & Project Completion Report

Smart-X Operational Intelligence Suite (Network Topology & Mesh Routing)

Student name: Lesego Masike

Student number: ST10058999

Module code: PROG7312 / AAPD7112

Referencing style: Harvard

1. Implementation Report

1.1 Compiling, running and using the application suite

The suite is a single .NET 9 solution with three projects: SmartX.Core (shared domain and data structures), SmartX.Api (an ASP.NET Core Minimal API hosting every engine), and SmartX.Client (a WPF desktop client). Prerequisites are the .NET 9 SDK and Windows 10/11.

To restore and build:

```
dotnet restore
dotnet build
```

To run, start the backend API first, then the client (or press F5 in Visual Studio 2022 with both projects set as startup projects):

```
dotnet run --project SmartX.Api --urls http://localhost:5101
dotnet run --project SmartX.Client
```

The client opens on a landing menu with all three pillars active: Sensor Data Ingestion (Part 1), Real-Time Command Stream (Part 2) and Network Topology & Mesh Routing (Part 3). Full step-by-step usage and API examples are in README.md.

1.2 Data-structure explanations (role, contribution and code)

Part 3 introduces Trees, Heaps and Graphs. Each was chosen to make a specific operation in the Network Topology feature efficient rather than merely functional.

AVL tree (self-balancing binary search tree) — historical range queries.

Each device's timestamped readings are stored in an AVL tree keyed by timestamp. A time-window query (for example, all readings between 14:00 and 16:00) then runs in O(log n + k), where k is the number of matches, instead of the O(n) cost of scanning a flat list. The AVL

property — sibling heights differ by at most one, restored by rotations on insert — keeps the height near log2(n), so queries stay fast as history grows (verified: 60 readings produce a height of 6). The range query prunes whole subtrees that fall outside the window:

```
private static void RangeQuery(Node? node, long lo, long hi, List<TimelinePoint> acc)
{
    if (node is null) return;
    if (node.Key > lo) RangeQuery(node.Left, lo, hi, acc);
    if (node.Key >= lo && node.Key <= hi)
        acc.Add(new TimelinePoint(new DateTimeOffset(node.Key, TimeSpan.Zero), node.Value));
    if (node.Key < hi) RangeQuery(node.Right, lo, hi, acc);
}
```

Binary min-heap — Prim's frontier.

Prim's algorithm repeatedly needs the cheapest edge leaving the growing tree. A binary min-heap provides that 'cheapest next' in O(log n) per push/pop. It is implemented from first principles with sift-up/sift-down over an array:

```
public T Pop()
{
    var top = _heap[0].Item;
    var last = _heap[^1];
    _heap.RemoveAt(_heap.Count - 1);
    if (_heap.Count > 0) { _heap[0] = last; SiftDown(0); }
    return top;
}
```

Graph (adjacency list) — the mesh model.

The physical mesh is modelled as a weighted, undirected graph: ESP32 nodes are vertices and wireless links are edges whose weight is a routing cost (approximately the distance between nodes). An adjacency list stores each node's neighbours, which is memory-efficient for the sparse mesh and gives fast neighbour iteration for traversal and MST construction.

Breadth-first / depth-first search — connectivity and down-node detection.

From the gateway, BFS (or DFS) visits every node reachable through live links; any node not visited is down or isolated. This answers 'which devices are connected, and which are down, and where must traffic hop instead?'. Traversal skips down vertices so it cannot route through a failed node:

```
while (queue.Count > 0)
{
```

```
    var cur = queue.Dequeue();
    order.Add(cur);
    foreach (var (to, _) in _adj[cur].OrderBy(e => e.To))
        if (IsUp(to) && visited.Add(to)) queue.Enqueue(to);
}
```

Minimum Spanning Tree (Prim's and Kruskal's) — power-efficient routing.

The MST is the cheapest set of links that still connects every live node — in IoT terms, the most power-efficient way for telemetry to hop through the mesh to the gateway. Prim's grows the tree from the gateway using the min-heap; Kruskal's sorts the edges and unions disjoint sets (union-find). Implementing both and confirming they return the same total weight (237.6 on the seeded mesh) is strong evidence of correctness. A device's route is then the unique path through the MST to the gateway. Prim's core loop:

```
while (!frontier.IsEmpty && inTree.Count < liveCount)
{
    var (from, to) = frontier.Pop();      // cheapest frontier edge (min-heap)
    if (inTree.Contains(to)) continue;
    mst.Add((from, to, WeightOf(from, to)));
    inTree.Add(to);
    foreach (var (next, nw) in _adj[to])
        if (IsUp(next) && !inTree.Contains(next)) frontier.Push((to, next), nw);
}
```

Contribution to efficiency: minimising total link cost means telemetry travels the lowest-energy multi-hop path — exactly what battery-powered mesh protocols such as Zigbee, Thread and ESP-NOW optimise for.

2. Project Completion Report

2.1 The completed Smart-X project

The three submissions form one coherent .NET 9 solution. SmartX.Core holds the shared domain and every data structure; SmartX.Api is a single Minimal API hosting all engines; SmartX.Client is a WPF client whose three pillars each talk to that API.

- **Part 1 (Ingestion):** registers devices, validates them, ingests multi-typed telemetry and surfaces an anomaly heat-strip. Concepts: generics (no boxing), operator overloading, recursion, arrays to collections.

- **Part 2 (Command Stream):** processes volatile data with a FIFO queue, a priority queue for critical alerts, a command undo stack, dictionaries, a sorted dictionary and a hash set, plus a predictive suggestion engine.
- **Part 3 (Network Topology):** visualises the mesh as a graph, runs BFS/DFS, computes the MST (Prim's and Kruskal's) for power-efficient routing, and answers historical range queries from an AVL tree.

All three pillars run together over the same backend, and navigation between them preserves each view's state.

2.2 Architectural and algorithmic challenges in Task 3, and how they were overcome

- **Integrating the C# backend with the WPF frontend:** The mesh graph is computed on the C# backend but drawn in WPF. This was solved by returning normalised layout coordinates (0–1) in the graph DTO, so the client only scales them to the canvas — the algorithms stay in one place and the UI stays thin. Every call is asynchronous so the UI never blocks on the network.
- **Rendering a graph:** WPF has no built-in graph control, so nodes and weighted edges are drawn directly on a Canvas, mapping normalised coordinates to pixels and redrawing on resize. Readability was solved with a radial layout (gateway centred) and a colour legend; MST and routes are shown by re-styling edges.
- **Guaranteeing MST correctness:** An MST is easy to get subtly wrong, so Prim's and Kruskal's were implemented independently and asserted to produce the same total weight. This cross-check caught an early bug where down nodes were still included in the tree.
- **Handling down nodes uniformly:** A failed node must be excluded consistently from traversal, MST and routing. An IsUp() check was centralised and used by every algorithm so a down node is never traversed or routed through.
- **Range querying without a database:** Fast time-window queries were needed over in-memory history without a database. An AVL tree was chosen so inserts stay balanced and range queries prune subtrees; the height was verified to stay logarithmic.

2.3 Key learnings

- Choosing the right data structure changes the complexity class of a feature — a list scan is O(n) while an AVL range query is O(log n + k). The Network Topology feature made this concrete.

- Implementing classic algorithms (Prim, Kruskal, BFS/DFS, AVL rotations) from first principles gave a deeper understanding than using library collections, and cross-checking two MST algorithms proved to be a practical correctness technique.
- Separating computation (the API) from presentation (WPF) behind a shared contract keeps a multi-module application maintainable, and reusing one instance of each view preserves state cheaply.
- Asynchronous HTTP with a thin client keeps a desktop UI responsive while it talks to a real backend.

3. Technology Recommendations & Future Scaling

The following technologies would enhance the Smart-X ecosystem. Each builds on the existing .NET stack rather than replacing it, which minimises risk and reuses existing skills.

- **CI/CD via GitHub Actions —** the solution already builds with dotnet build, so a workflow that runs dotnet build and tests on every push gives automatic regression checks and reproducible releases, using first-class actions/setup-dotnet with no new runtime.
- **Containerisation and orchestration —** the API already ships with a Dockerfile; orchestration (Kubernetes or Azure Container Apps) would let the gateway scale horizontally as the mesh grows to thousands of nodes, with health checks and rolling updates. ASP.NET Core runs unchanged in Linux containers.
- **A managed time-series / distributed database —** the current store is in-memory; a managed time-series or distributed database (Azure SQL, Cosmos DB or TimescaleDB) would persist history across restarts and scale beyond one process, and the AVL/range-query concept maps naturally onto an indexed time range. EF Core / ADO.NET integrate cleanly with .NET.
- **An LLM API to summarise telemetry alerts —** the dashboard already produces alerts and insights; an LLM endpoint could turn a day's anomalies into a plain-language operator brief, improving engagement. It is a simple HTTPS call from the existing API, mirroring how the client already calls the backend.
- **MQTT ingestion and Home Assistant interoperability —** real ESP32 devices speak MQTT; adding an MQTT broker (for example MQTTnet in .NET) would let the same API ingest real hardware and interoperate with platforms such as Home Assistant, moving the simulation toward production.

Each recommendation is chosen for direct compatibility with the .NET ecosystem and for building on components that already exist in the project, which keeps the security, scalability and performance gains achievable at low risk.

References

Albahari, J. and Albahari, B., 2022. C# 10 in a Nutshell: The Definitive Reference. Sebastopol: O'Reilly Media.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2009. Introduction to Algorithms. 3rd ed. Cambridge, MA: MIT Press.

Microsoft, 2024. Collections and Data Structures. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/standard/collections/ [Accessed 25 September 2026].

Microsoft, 2024. Minimal APIs overview. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis [Accessed 25 September 2026].

Microsoft, 2024. Windows Presentation Foundation (WPF) documentation. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/ [Accessed 25 September 2026].

Sedgewick, R. and Wayne, K., 2011. Algorithms. 4th ed. Boston: Addison-Wesley.